

Observation Task 2

Object-Oriented Programming in Java

2 Hours

Objective

To understand and implement the core Object-Oriented Programming (OOP) concepts in Java. Concepts Covered

- Classes and Objects
- Constructors
- Encapsulation
- Inheritance
- Method Overloading
- Method Overriding
- Polymorphism
- Abstraction
- Interfaces

Task Description

Develop a "**Library Management System**" to demonstrate Object-Oriented Programming concepts.

Part A: Classes and Objects

Create a class named Book containing:

- Book ID
- Book Name
- Author
- Price

Create objects and display their details.

Part B: Constructors and Encapsulation

- Create both default and parameterized constructors.
- Declare data members as private.
- Access them using Getter and Setter methods.

Part C: Inheritance

Create the following classes:

- Person
- Student (inherits Person)
- Faculty (inherits Person)

Display the inherited properties using objects.

Part D: Polymorphism

Implement:

- Method Overloading for calculating the area of different shapes.
- Method Overriding by creating a Vehicle class and overriding the display() method in Car and Bike classes.

Part E: Abstraction and Interfaces

- Create an abstract class Shape containing an abstract method draw().
- Implement the draw() method in Circle and Rectangle classes.
- Create an interface Printable and implement it in a class named Report.

1. Steps of the Program

Part A – Encapsulation (Book)

1. Create a Book class.
2. Declare bookId, bookName, author, and price as **private**.
3. Create a constructor to initialize all book details.
4. Create **getter methods** to read the private data.
5. Create **setter methods** to modify the private data.
6. Create a display() method to display book details.
7. In main(), create a Book object and display its details.

Part B – Inheritance (Person, Student, Faculty)

8. Create a Person superclass with name, age, and display().
9. Create a Student class that extends Person.
10. Add course to the Student class.
11. Use super.display() to display the parent class details.
12. Create a Faculty class that also extends Person.

13. Add department to the Faculty class.
 14. Create Student and Faculty objects and display their details.
-

Part C – Method Overloading (Area)

15. Create an Area class.
 16. Create areacircle(double radius) to calculate circle area.
 17. Create arearectangle(int length, int breadth) to calculate rectangle area.
 18. Call both methods using an Area object.
 19. This demonstrates **method overloading** because the methods have different parameters.
-

Part D – Method Overriding (Vehicle, Car, Bike)

20. Create a Vehicle class with a display() method.
 21. Create Car that extends Vehicle.
 22. Override the display() method in Car.
 23. Create Bike that extends Vehicle.
 24. Override the display() method in Bike.
 25. Create Vehicle references pointing to Car and Bike objects.
 26. Call display() using both references.
 27. The overridden methods of Car and Bike are executed.
-

Part E – Abstraction (Shape)

28. Create an abstract class Shape.
29. Declare an abstract method draw().
30. Create Circle that extends Shape.
31. Implement the draw() method in Circle.
32. Create Rectangle that extends Shape.
33. Implement the draw() method in Rectangle.

34. Create Shape references for Circle and Rectangle.
 35. Call draw() to display the appropriate shape.
-

Part F – Interface (Printable)

36. Create an interface Printable.
 37. Declare the print() method inside the interface.
 38. Create a Report class that implements Printable.
 39. Implement the print() method in Report.
 40. Create a Printable reference pointing to a Report object.
 41. Call print() to display the report message.
 42. **End the program.**
-

2. Overall Algorithm

1. **Start**
2. Create a Book object using the constructor.
3. Display the book details using the display() method.
4. Create a Student object.
5. Assign name, age, and course.
6. Display student details using inherited properties and super.display().
7. Create a Faculty object.
8. Assign name, age, and department.
9. Display faculty details.
10. Create an Area object.
11. Call areacircle() to calculate and display the circle area.
12. Call arearectangle() to calculate and display the rectangle area.
13. Create a Car object using a Vehicle reference.
14. Create a Bike object using a Vehicle reference.
15. Call display() for both objects to demonstrate **method overriding**.

16. Create a Circle object using a Shape reference.
17. Create a Rectangle object using a Shape reference.
18. Call draw() for both objects to demonstrate **abstraction**.
19. Create a Report object using a Printable reference.
20. Call print() to demonstrate the **interface**.
21. **Stop**.

Code :

```
class Book {  
    private int bookId;  
    private String bookName;  
    private String author;  
    private double price;  
    Book(int id, String name, String auth, double pr) {  
        bookId = id;  
        bookName = name;  
        author = auth;  
        price = pr;  
    }  
    public int getBookId() {  
        return bookId;  
    }  
    public void setBookId(int id) {  
        bookId = id;  
    }  
    public String getBookName() {  
        return bookName;  
    }  
}
```

```

public void setBookName(String name) {
    bookName = name;
}
public String getAuthor() {
    return author;
}
public void setAuthor(String auth) {
    author = auth;
}
public double getPrice() {
    return price;
}
public void setPrice(double pr) {
    price = pr;
}
public void display() {
    System.out.println("Book ID  : " + bookId);
    System.out.println("Book Name : " + bookName);
    System.out.println("Author   : " + author);
    System.out.println("Price    : " + price);
}
}

```

// Part C: Inheritance

```

class Person {
    String name;
    int age;
    void display() {
        System.out.println("Name : " + name);
    }
}

```

```

        System.out.println("Age : " + age);
    }
}

class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course : " + course);
    }
}

class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department : " + department);
    }
}

```

// Part D: Method Overloading

```

class Area {
    void areacircle(double radius) {
        System.out.println("Circle Area = " + (3.14 * radius * radius));
    }
    void arearectangle(int length, int breadth) {
        System.out.println("Rectangle Area = " + (length * breadth));
    }
}

```

// Part D: Method Overriding

```

class Vehicle {

```

```
void display() {  
    System.out.println("This is a vehicle.");  
}  
}
```

```
class Car extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a car.");  
    }  
}
```

```
class Bike extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a bike.");  
    }  
}
```

// Part E: Abstraction

```
abstract class Shape {  
    abstract void draw();  
}
```

```
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}
```

```
class Rectangle extends Shape {  
    @Override
```



```

void draw() {
    System.out.println("Drawing a Rectangle");
}
}

// Part E: Interface
interface Printable {
    void print();
}

class Report implements Printable {
    @Override
    public void print() {
        System.out.println("Printing Report...");
    }
}

public class Library {
    public static void main(String[] args) {
        System.out.println(" BOOK DETAILS");

        Book b = new Book(101,"Java Programming","James Gosling",500);
        b.display();

        System.out.println("\n INHERITANCE");

        Student s = new Student();
        s.name = "Bob";
        s.age = 25;
        s.course = "Engineering";
        s.display();

        System.out.println("\n faculty");

        Faculty f = new Faculty();
        f.name = "Dr. Bob";
    }
}

```

```

f.age = 45;
f.department = "Engineering";
f.display();
System.out.println("\n METHOD OVERLOADING");

Area a = new Area();
a.areacircle(5.0);
a.arearectangle(10, 20);
System.out.println("\n METHOD OVERRIDING");

Vehicle v1 = new Car();
Vehicle v2 = new Bike();

v1.display();
v2.display();

System.out.println("\n ABSTRACTION ");

Shape sh1 = new Circle();
Shape sh2 = new Rectangle();

sh1.draw();
sh2.draw();

System.out.println("\n INTERFACE ");

Printable p = new Report();

p.print();
}
}

```

Output :

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-  
eptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\  
\jdt_ws\jdt.ls-java-project\bin' 'Library'
```

BOOK DETAILS

```
Book ID   : 101  
Book Name : Java Programming  
Author    : James Gosling  
Price     : 500.0
```

INHERITANCE

```
Name : Bob  
Age  : 25  
Course : Engineering
```

faculty

```
Name : Dr. Bob  
Age  : 45  
Department : Engineering
```

METHOD OVERLOADING

```
Circle Area = 78.5  
Rectangle Area = 200
```

METHOD OVERRIDING

```
This is a car.  
This is a bike.
```

ABSTRACTION

```
Drawing a Circle  
Drawing a Rectangle
```

INTERFACE

```
Printing Report...  
PS C:\Users\yabhi\abhi.java>
```